RMS Prop

→ Core Idea → Keep a moving average of squared gradients instead of accumulating all of them.

⇒ Key Innovations –
- uses EMA (exponential moving avg.) of squared gradients
- recent gradients will have more influence
- past gradients decay exponentially
- prevents the AdaGrad's problem of aggressive decay

⇒ How RMS Prop addresses AdaGrad's limitations?

- maintains non-0 LR.
- adapts to the behaviour of the recent gradients
- better for non-convex optimization problems
- suitable for online / mini-batch learning.

Eg. while training a neural network, early times the gradient will be large & mostly unstable. Later steps, gradient will be small but stable. Because of large early gradients, the LR will get permanently reduced. So the goal is to forget the old gradients gradually, and only focus on recent behaviour.

⇒ EMA (Exponential Moving Avg.)

$$E[g^2]_t = \beta \cdot E[g^2]_{t-1} + (1-\beta)\{g^2\}_t$$

$\beta \approx 0.9$, controls the decay rate

- recent wts g_t^2 have wt. $(1-\beta)$, past avg. $E[g^2]_{t-1}$ has wt. β.
- gradients from $\frac{1}{(1-\beta)}$ steps have the most influence.

⇒ Algo.

init θ_0, η, $\beta(0.9)$, $E[g^2]_0 = 0$, $\varepsilon(10^{-8})$

for $t = 1$ to T,

$g_t = \nabla_\theta J(\theta)$ ⇒ compute gradients

$$E[g^2]_t = \beta E[g^2]_{t-1} + (1-\beta) g_t \odot g_t$$

↳ update the moving avg.

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{E[g^2]_t + \varepsilon}} \odot g_t$$

→ update the params

- earlier, in AdaGrad,

$G_t = G_{t-1} + g_t^2$ (unbounded accumulation)

now, in RMSProp

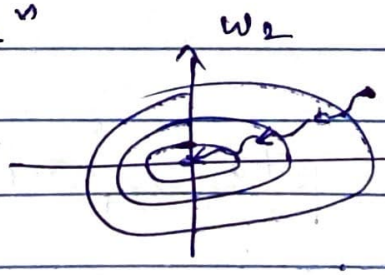
$$E[g^2]_t = \beta E[g^2]_{t-1} + (1-\beta) g_t^2$$

(this expression is bounded by $\frac{1}{1-\beta} \max_\tau g_\tau^2$)

↓

now the LR depends on a wtd. avg. of recent squared gradients

⇒ Visualiza"



maintains adaptive learning throughout

better convergence in non-convex landscapes

→ RMSProp vs AdaGrad

- gradient accum. G_t	$G_t = \sum_{t=1}^{T} g_t^2$	$E[g^2]_t = \beta E[g^2]_{t-1} + (1-\beta) g_t^2$
- LR	dec. ↓	can ↑ or ↓
- long training	stops midway	continues learning
- memory	all equally	recent gradients
- HPT	η	η, β
- best for	sparse, convex problems	non-convex, online, mb learning

→ Limitations → still depends on initial LR.

1. LR sensitivity - large η (instability), small η (small convergence), manual
2. units inconsistency - g_t & LR have diff. units.

$$\Delta\theta = \frac{\eta}{\sqrt{E[g^2]_t}} g_t$$

3. HPT - tune both η & β. diff. problems req. diff. values.